

Projet d'introduction à l'IA

Objectif général

L'objectif de ce projet est de vous initier à la modélisation et à la résolution de problèmes d'intelligence artificielle à travers la programmation de **trois jeux** en C++, avec une **interface en mode console**.

Chaque jeu devra être formulé comme un **problème de recherche (ou d'exploration)**, en identifiant clairement les éléments suivants :

- **Etat initial** : description de la configuration du jeu.
- **Joueur(s)** : identification du joueur qui doit jouer dans un état donné.
- **Actions possibles** : ensemble des coups autorisés dans un état donné.
- **Résultat (s, a)** : état obtenu après l'application d'une action a à l'état s .
- **Test terminal** : indique si la partie est terminée ou non.
- **Fonction d'évaluation** : attribue une valeur numérique à un état terminal pour un joueur donné.

Projet 1 : Jeu de taquin (8 pièces)

Le jeu de taquin est composé d'un **plateau de 3×3 cases**, dont **8 cases contiennent des pièces numérotées** et **une case est vide**. Une pièce adjacente à la case vide peut être déplacée vers celle-ci.



Figure 1 : Exemple d'un état initial et d'un état final du jeu de taquin

Objectif

Atteindre un **état final donné** à partir d'un **état initial donné**, en effectuant une suite de déplacements valides.

Actions possibles

Les mouvements autorisés sont : **Gauche, Droite, Haut, Bas**.

Travail demandé

- Implémenter le jeu en C++.
- Résoudre le problème à l'aide de l'algorithme A*.
- Utiliser :
 - La **distance de Manhattan** comme fonction heuristique.
 - Le **coût du chemin** correspondant à la profondeur (ou longueur) du chemin depuis l'état initial jusqu'à l'état courant.

Projet 2 : Le Jeu du virus

Le jeu du virus se joue sur une **grille carrée** (généralement 7×7). Au début de la partie :

- Deux pions blancs et deux pions noirs sont placés sur des **coins opposés** du plateau.

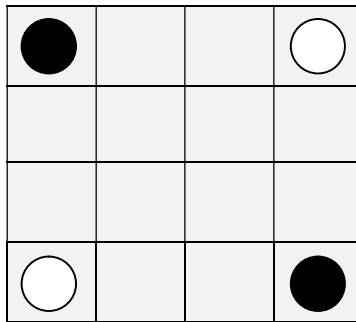


Figure 2 : Etat initial du jeu de virus

Règles du jeu

- A chaque tour, un joueur place un pion de sa couleur sur l'une des huit cases voisines d'un de ses pions existants.
- La case choisie doit être **vide**.
- Après le placement, **tous les pions adverses voisins** du pion posé changent de couleur et deviennent de la couleur du joueur.
- La partie se termine lorsque **toutes les cases du plateau sont remplies**.

Condition de victoire

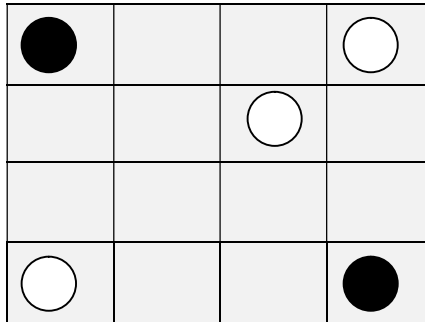
Le joueur qui possède **le plus grand nombre de pions** à la fin de la partie est déclaré gagnant.

Travail demandé

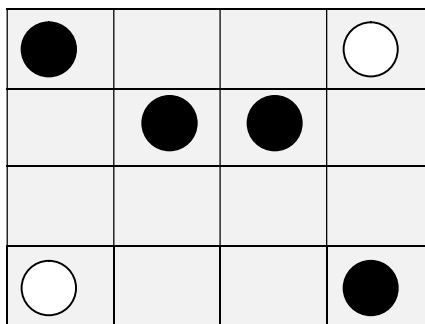
- Implémenter le jeu en C++ basé sur l'algorithme **Minimax** et l'optimisation **Alpha-Bêta**.
- Modéliser correctement les états, les actions et les règles de transformation des pions.
- Le jeu devra proposer **trois niveaux de difficulté** :
 - Débutant
 - Moyen
 - Expert

Exemple de coups :

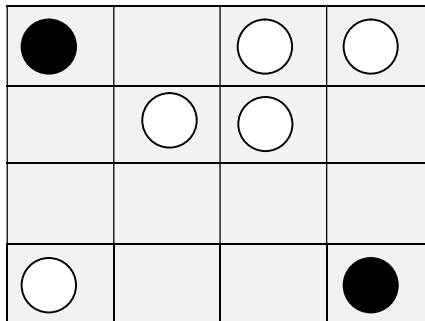
Etape 1 : Coup blanc



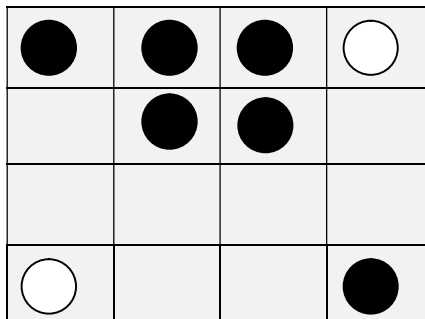
Etape 2 : Coup noir



Etape 3 : Coup blanc



Etape 4 : Coup noir

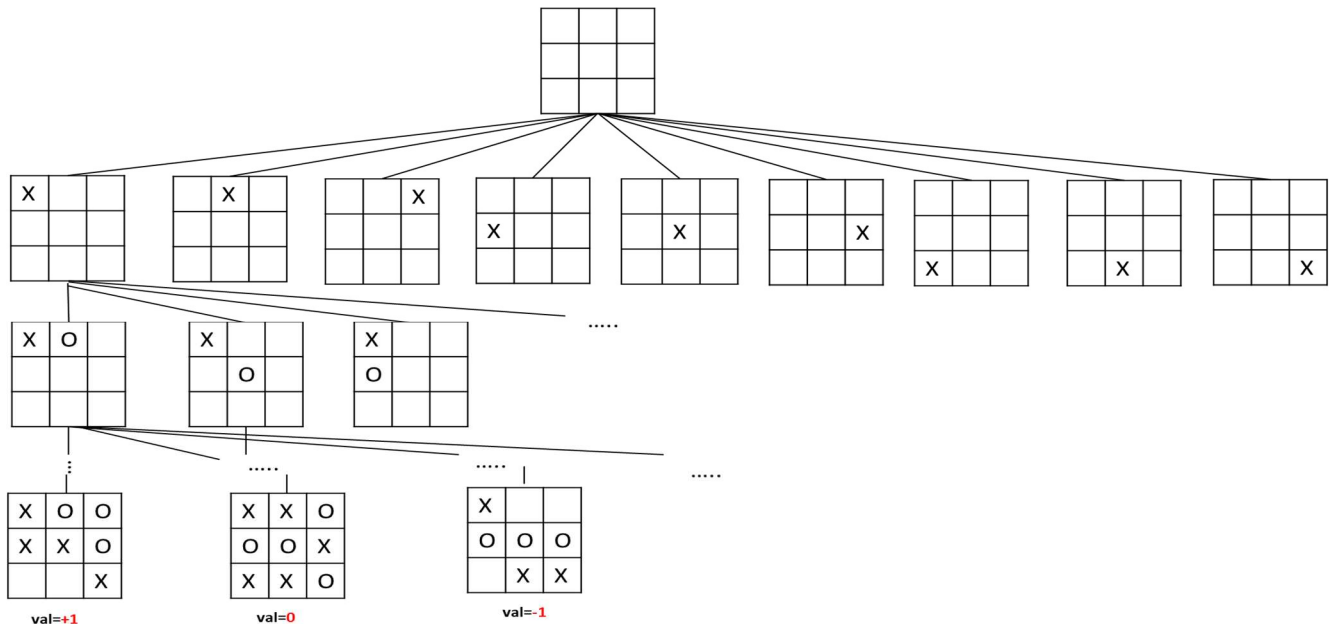


Projet 3 : Le jeu du Morpion ((Tic-Tac-Toe)

Le morpion est un jeu à deux joueurs :

- Max (croix)
- Min (Rond)

Les joueurs jouent alternativement sur une **grille 3×3** jusqu'à atteindre un état terminal. Au départ, le joueur **MAX** dispose de neuf coups possibles. Au cours d'une partie, **MAX** et **MIN** placent alternativement une marque, jusqu'à atteindre un nœud feuille qui correspond à un état terminal dans lequel l'un des deux joueurs a aligné trois marques ou dans lequel toutes les cases sont remplies. Le nombre placé sur chaque nœud feuille indique la valeur d'utilité (évaluation du gain) de l'état terminal du point de vue de MAX. On suppose que les valeurs élevées sont bonnes pour MAX et mauvaises pour MIN.



Règles du jeu :

- Le jeu s'arrête dès qu'un joueur aligne **trois pions** sur une ligne, une colonne ou une diagonale, ou lorsque la grille est pleine.
- La différence entre le nombre de pions des deux joueurs ne doit jamais dépasser **un pion**.
- Les deux joueurs ne peuvent pas gagner simultanément.

Travail demandé :

1) Analyse des configurations

Écrire un programme C++ permettant de :

- Enumérer les configurations possibles sans contrainte.
- Identifier les **configurations valides**, en respectant les règles du jeu.
- Calculer le **nombre de configurations valides à chaque tour**.

- Calculer le **nombre de configurations gagnantes à chaque tour**.

2) *Jeu intelligent*

Écrire un programme C++ basé sur :

- L'algorithme **Minimax**
- L'optimisation **Alpha-Bêta**
- Le jeu devra proposer **trois niveaux de difficulté** :
 - Débutant
 - Moyen
 - Expert

Consignes générales

- Le code doit être clair, bien structuré et commenté.
- Une attention particulière sera portée à la **modélisation des états**, à la **qualité des algorithmes** et à la **lisibilité du programme**.
- Toute initiative supplémentaire (optimisation, affichage amélioré, statistiques, etc.) sera valorisée.

Bon courage et bon travail !